Gomez Guzman
Jonathan Ulises.

04/10/2026

Universidad de Guadalajara.

Centro Universitario de Tonalá.



Programación de Servicios Web.

Actividad 3.2: Diseño de Servicios Web RESTful.

Equipo 06:

Gomez Guzman Jonathan Ulises.

Gomez Salcedo Sonia.

Diseño.

Se identificaron 16 futuras tablas. Las tablas técnicas de Laravel como **USERS, CACHE** y **JOBS** no serán publicadas para privacidad, la API empleará JSON; los archivos binarios de imagen tendrán un endpoint multipart separado.

Propondremos nueve recursos con ciclo **CRUD,** tres recursos auxiliares con operaciones restringidas y cuatro asociaciones anidadas. Done las excepciones evitan alterar auditorias, falsificar pagos o vincular cuentas sociales sin demostrar su titularidad de este mismo para el huerto o producto.

Recursos y entidades.

Cada recurso CRUD tiene listado, consulta individual, creación, reemplazo PUT, cambio parcial PATCH, borrado lógico, restauración y borrado definitivo. La papelera usa el listado con eliminados=solo. Las asociaciones no se presentan como módulos CRUD independientes.

Especificaciones.

Base local propuesta: http://127.0.0.1:8000/api/v1. Todos los endpoints de las tablas se agregan a esta base. En producción será obligatorio HTTPS.

La interfaz web aun existente conserva sus rutas sin el /api/v1 propuesto.

Encabezados (Headers) y Autenticación Comunicación en JSON: Dile siempre al servidor que esperas recibir JSON agregando Accept: application/json. Solo incluye el encabezado Content-Type: application/json cuando tu petición envíe un cuerpo (body) con datos JSON (por ejemplo, en POST, PUT o PATCH). Todas las respuestas que contengan información vendrán en formato application/json.

- Respuestas sin contenido (204 No Content): Cuando el servidor responda con un estado **204** (por ejemplo, tras eliminar un registro con éxito), la respuesta no contendrá ningún cuerpo ni mensaje JSON; vendrá completamente vacía.
- Manejo de Tokens: Envíalo mediante el encabezado Authorization: Bearer . Omite este encabezado en rutas públicas o en el endpoint de login.
- Seguridad con el Token: Si intentas acceder a una ruta protegida sin un token válido, la API responderá con un error **401** Unauthorized e incluirá el encabezado WWW-

Authenticate: Bearer. Nunca pases tokens en los parámetros de la URL ni los guardes en logs del sistema, ya que expone credenciales en texto plano.

Identificadores (IDs) y Jerarquía de RecursosTipos de ID: La mayoría de los recursos usan enteros positivos ($1, 2, 3...$), salvo las notificaciones, que utilizan **UUID** (por ejemplo, 123e4567-e89b-12d3-a456-426614174000).

- Validación de IDs en la URL: Si el ID tiene un formato incorrecto (un texto donde va un entero o un UUID mal escrito), recibirás un **400** Bad Request. Si el formato es válido pero el registro no existe o está fuera de alcance, recibirás un **404** Not Found.
- Jerarquía Padre-Hijo: En rutas anidadas (ej. /clientes/5/pedidos/12), el objeto hijo debe pertenecer estrictamente al padre especificado en la URL. Si el pedido 12 existe, pero pertenece a otro cliente, la API responderá con **404** Not Found para evitar fuga de información.
- Inmutabilidad y Validación del Cuerpo: No puedes intentar modificar la clave primaria (id) enviándola en el cuerpo de la petición. La API es estricta: si envías campos no reconocidos, propiedades calculadas, datos de auditoría (como created_at) o atributos para los que no tienes permisos, la API responderá con **422** Unprocessable Entity en lugar de ignorarlos en silencio.

Fechas y Horas: Siguiran el estándar **ISO 8601** en UTC (YYYY-MM-DDTHH:mm:ssZ, por ejemplo 2026-10-04T15:00:00Z). Si el campo es solo una fecha sin hora, usa el formato YYYY-MM-DD.

Números Decimales: Los montos o decimales precisos se representan como cadenas de texto (strings) de precisión fija (ej. "99.90" en lugar de 99.9). Esto evita los errores de redondeo típicos de la coma flotante en JSON.

Valores Nulos y Listas Vacías:

- Si un dato no existe, usa la palabra reservada null en JSON, nunca un string vacío ("").
- Si un listado o colección no tiene elementos, la API siempre devolverá un arreglo vacío [], nunca null.

Parámetros de Búsqueda (Query Parameters) en Listados

Paginación Estándar:

- page: Número de página (entero > 1, por defecto 1).
- per_page: Cantidad de resultados por página (entero entre 1 y 50, por defecto 10).
- Si pides una página que excede el total disponible, la API responderá con 200 OK devolviendo una lista vacía [] en el atributo de datos.

Manejo de Borrado Lógico (Soft Delete): El parámetro "**eliminados**" acepta solo dos valores: no (por defecto) y solo.

- Usar eliminados=solo permite listar únicamente los registros marcados como eliminados. Esto requiere que tu usuario tenga un permiso específico sobre el recurso terminado en. papelera (por ejemplo, usuarios.papelera).
- Este parámetro solo está disponible en endpoints cuyos recursos implementen borrado lógico.

Validaciones Strict: Pasar parámetros de consulta desconocidos o no documentados devolverá un error **422** Unprocessable Entity.

GET Operaciones Individuales y Ciclo de Vida (GET, Restaurar, Eliminar Definitivo)

GET de un solo recurso: Por defecto solo busca registros activos. Sin embargo, soporta el parámetro eliminado=solo para consultar un registro específico borrado, siempre que cuentes con el permiso de papelera.

Restricción de Parámetros en otros Métodos: Los métodos como POST, PUT, PATCH o DELETE no aceptan parámetros de consulta en la URL (Query Parameters).

Rutas de Restauración y Eliminación Definitiva:

- Estas rutas están diseñadas para operar exclusivamente sobre registros eliminados.
- Si intentas ejecutar una acción de restauración o borrado definitivo sobre un registro que aún está activo (o en un estado incompatible), la API rechazará la solicitud devolviendo un **409** Conflict.

Borrado Lógico y Estado Operativo

Eliminado significa eliminado_en distinto de null. No es igual al estado operativo inactivo. El listado administrativo muestra no eliminados, incluidos inactivos; el catálogo público filtra además estado activo o disponible. Un DELETE lógico cambia solo eliminado_en; no borra relaciones ni archivos. Restaurar limpia ese campo después de validar padres vigentes, unicidad y permisos.

PUT reemplaza todos los campos editables del esquema de actualización:

Obligatorios deben estar presentes y opcionales omitidos pasan a null o al valor por defecto documentado. PATCH modifica solo los campos enviados y requiere al menos uno. ID, fechas, existencias calculadas, imágenes y credenciales conservadas no forman parte del reemplazo PUT. Contraseña de usuarios es una excepción expresa: omitida se conserva, null se rechaza. PUT no crea registros inexistentes en esta API.

Autenticacion y permisos.

Proponemos Laravel Sanctum con tokens Bearer, se configurará el proveedor con el modelo Usuario y sus campos correo y contraseña, sin confundirlo con User de la plantilla inicial. La contraseña se verifica contra un **hash** y jamás se devuelve. Los tokens expiran a las 8 horas por decisión del proyecto y se revocan al cerrar sesión, desactivar o eliminar al usuario. Para una futura SPA propia con cookies deberá documentarse otro flujo con protección CSRF.

Rol	Alcance
Administrador	Administra recursos de negocio; consulta auditoría; purga solo cuando la integridad lo permite.
Productor	Huertos y productos propios; miembros de huertos propios; inventario propio; carrito, lista, órdenes y notificaciones personales.
Cliente	Carrito, lista, órdenes y notificaciones personales; catálogo público.
Visitante	Únicamente catálogo público e inicio de sesión.

El servidor comprueba rol, permiso y propiedad del objeto en cada solicitud. Los identificadores de usuario del cuerpo no permiten suplantar a otro titular.

Permisos CRUD por recurso:

Listar, ver, crear, editar, eliminar, papelera, restaurar y purgar. Administrador tiene estos permisos en los nueve recursos, sujetos a reglas de integridad. Productor tiene los siete primeros excepto purgar en huertos y productos propios; administrador es el único que purga. Titulares tienen: listar/ver/crear/editar/eliminar/papelera/restaurar en carritos, listas-deseos y ordenes propios, respetando las restricciones financieras. En notificaciones los titulares solo listar/ver y leer. Roles, usuarios y transacciones son exclusivamente administrativos en esta versión.

Respuestas y errores comunes

Las fichas de endpoints heredan estas convenciones y el esquema de su recurso. Las respuestas individuales usan data como objeto; las colecciones usan data como arreglo. La creación incluye Location con la URL del nuevo recurso. Las mutaciones con 200 agregan message.

Ejemplo:


```
{
 "data": [
  {
   "id": 1,
   "nombre": "Huerto Las Flores"
  }
 ],
 "meta": {
  "current_page": 1,
  "per_page": 10,
  "total": 1,
  "last_page": 1
 },
 "links": {
  "prev": null,
  "next": null
 }
}
```


En listados, cada elemento utiliza el esquema de salida del recurso; el ejemplo anterior abrevia campos. Las URLs de links conservan los filtros. En la consulta individual se incluyen relaciones descriptivas sin ciclos; contraseñas y secretos siempre se excluyen.

Código	Uso y Mensaje
400	SOLICITUD_INVALIDA: El JSON o el identificador no tiene un formato válido.
401	AUTENTICACION_REQUERIDA: Inicia sesión con credenciales o un token válido.
403	PERMISO_INSUFICIENTE: No tienes permiso para realizar esta operación.
404	REGISTRO_NO_ENCONTRADO: No se encontró el registro solicitado.
409	CONFLICTO_DE_INTEGRIDAD: No se puede eliminar porque existen registros relacionados.
422	DATOS_INVALIDOS: Revisa los campos indicados.
500	ERROR_INTERNO: No fue posible completar la operación. Intenta más tarde.
200 OK	Consulta, edición, borrado lógico o restauración exitosa con JSON.
201 Created	Nuevo recurso creado con JSON y Location.
204 No Content	Borrado físico o cierre de sesión exitoso, sin cuerpo.
413 Content too large	Imagen que supera el límite aceptado.
415 Unsupported Media Type	Content-Type no admitido.
429 Too Many Requests	Límite de intentos; incluir Retry-After.

Ejemplos:

```
{
  "message": "Revisa los campos indicados.",
```

```
 "error": {
  "code": "DATOS_INVALIDOS",
  "fields": {
   "nombre": [
    "El nombre es obligatorio."
   ],
   "correo": [
    "El correo no tiene un formato válido."
   ]
  }
 }
}
```

```
{
  "message": "No se puede eliminar el huerto porque tiene productos asociados.",
  "error": {
    "code": "CONFLICTO_DE_INTEGRIDAD",
    "fields": {},
    "details": {
      "productos": 2
    }
  }
}
```


422 incluye campos obligatorios, límites y FK de entrada inexistente o no vigente. **409** incluye relaciones que bloquean purga, duplicados que violan unicidad, restauraciones incompatibles y estados financieros no editables. **400** se reserva para sintaxis/formato, no sustituye a **422**. Para login, **401** usa CREDENCIALES_INVALIDAS y mensaje genérico, sin revelar si existe el correo. **500** no expone SQL, rutas físicas, trazas ni contraseñas. El servidor registra el detalle de forma privada.

Login (POST /auth/login)

Este endpoint se encarga de validar las credenciales de acceso del usuario y emitir su token de autenticación para que pueda consumir el resto de la API.

Acceso y Seguridad

Sin Autenticación: Es una ruta pública; no requiere enviar el encabezado Authorization.

Elegibilidad del Usuario: Solo pueden autenticarse usuarios activos/vigentes cuyo rol tenga acceso asignado al sistema.

Permisos: No se requiere ningún permiso específico previo.

Petición (Request)

Parámetros de URL y Query: No se acepta ningún parámetro en la ruta ni en la consulta (?param=val).

Encabezados (Headers):

Accept: application/json

Content-Type: application/json

Cuerpo de la Petición (JSON): Contiene tres campos obligatorios:

Ejemplos:

```
{
  "correo": "ana@example.com",
  "contrasena": "EjemploSeguro2026!",
  "dispositivo": "Navegador de prueba"
}
```

Si las credenciales y los datos son correctos, la API responde con un objeto raíz data que incluye las propiedades del token emitido y la información básica del usuario (excluyendo datos sensibles como contraseñas o hashes de seguridad). El rol del usuario se devuelve como un objeto anidado:

```
{
  "data": {
    "token": "TOKEN_DE_EJEMPLO_NO_VALIDO",
```

```
  "token_type": "Bearer",
  "expires_at": "2026-10-04T23:00:00Z",
  "usuario": {
    "id": 2,
    "nombres": "Ana",
    "apellidos": "López",
    "rol": {
      "id": 2,
      "nombre": "productor"
    }
  }
 }
}
```


Módulo de Autenticación GET /auth/me

Obtener Perfil del Usuario (GET /auth/me) obteniendo una respuesta exitosa que devuelve el perfil y la lista de permisos activos del usuario:


```
{
 "data": {
  "id": 2,
  "nombres": "Ana",
  "apellidos": "López",
  "correo": "ana@example.com",
  "rol": {
```

```
    "id": 2,
    "nombre": "productor"
  },
  "permisos": [
    "huertos.listar",
    "huertos.ver",
    "huertos.crear",
    "huertos.editar",
    "huertos.eliminar",
    "huertos.papelera",
    "huertos.restaurar"
  ]
 }
}
```


- Cerrar sesión (POST /auth/logout): Invalida de inmediato únicamente el token desde el que haces la petición. No borra otras sesiones abiertas en otros dispositivos.
- Seguridad en respuestas: Ni el inicio de sesión ni el endpoint /auth/me guardan copia en caché (Cache-Control: no-store). Además, el login bloquea al usuario si detecta más de 5 intentos fallidos por minuto desde la misma IP.
- Acceso restringido: No existen rutas públicas para asignarse roles de mayor privilegio ni para crear usuarios administradores.

Los Módulos Principales (CRUDs)

Todos los recursos principales de la API comparten una estructura idéntica de 8 acciones (Listar, Ver, Crear, Reemplazar completo, Editar parcial, Enviar a papelera, Restaurar y Eliminar definitivo).

Roles

Acceso: Exclusivo de administradores.

Reglas clave: El nombre es único y no puede exceder 50 caracteres. No puedes eliminar un rol si aún hay usuarios asignados a él (incluso en papelera). Jamás se puede modificar o borrar el rol de Administrador si esto dejara al sistema sin un administrador activo.

Petición de creación (POST /roles):


```
{
  "nombre": "productor",
  "descripcion": "Gestiona sus huertos y productos"
}
```

```
{
  "message": "Registro creado correctamente.",
  "data": {
    "id": 1,
    "nombre": "productor",
    "descripcion": "Gestiona sus huertos y productos",
    "creado_en": "2026-10-04T15:00:00Z",
    "actualizado_en": "2026-10-04T15:00:00Z",
    "eliminado_en": null
  }
}
```


Usuarios

Acceso: Exclusivo de administradores.

Reglas clave: El correo es único. Las contraseñas se piden al crear (entre 12 y 128 caracteres), pero nunca se devuelven. Al editar un usuario, si no envías el campo contraseña, se mantiene intacta. No puedes eliminar tu propio usuario ni al último administrador. Si desactivas o envías a papelera a un usuario, sus tokens se revocan automáticamente.

Huertos

Acceso: Administradores y productores (estos últimos solo gestionan sus propios huertos).

Reglas clave: Requiere nombre, dirección y un propietario activo. Las coordenadas (latitud y longitud) y el área son opcionales. Un productor no puede reasignar la propiedad de su huerto a otra persona. Si el huerto tiene miembros o productos asociados, no se puede eliminar de forma definitiva.

Productos

Acceso: Administradores y el productor dueño del huerto.

Reglas clave: El stock o existencias no se envía manualmente: el servidor lo calcula sumando y restando los movimientos de inventario. Puedes definir precios decimales, unidades (kg, pieza, etc.) y rangos de fechas de cosecha/disponibilidad. Si un producto ya figuró en una orden o carrito, su eliminación definitiva queda bloqueada para proteger el historial financiero.

Carritos y Listas de Deseos

Acceso: Administradores y el usuario dueño (titular).

Reglas clave: Cada usuario tiene una lista y un carrito único. No puedes cambiar el dueño de un carrito o lista existente. El contenedor solo se puede borrar de forma permanente si está completamente vacío. Para modificar sus productos se usan sus endpoints anidados específicos.

Órdenes de Compra

Acceso: Administradores y el comprador titular.

Reglas clave: Tú envías los productos y cantidades; el servidor calcula precios, subtotales y totales para evitar manipulaciones. Al crearse, se asigna un folio único (ej. ORD-000001). Solo puedes editar las notas o los ítems si la orden está en estado pendiente y no tiene pagos asociados. Una vez pagada, la orden se vuelve inmutable: no se puede alterar ni borrar.

Transacciones (Pagos)

Acceso: Exclusivo de administradores.

Reglas clave: Solo se gestionan manualmente en modo de prueba o para compras pendientes. El monto no puede superar el saldo pendiente de la orden. Nunca se guardan números de tarjeta de crédito, CVV o claves secretas. Un pago confirmado es completamente inmutable.

Notificaciones

Identificador especial: Utilizan formato UUID (cadenas largas tipo 96cc3d48-...) en lugar de números enteros.

Reglas clave: Los administradores las crean y envían; los usuarios destinatarios solo las consultan y las marcan como leídas mediante la ruta PATCH /notificaciones/{id}/lectura (la cual registra la fecha y hora de lectura en el servidor).

Para Sanctum se requieren instalación/configuración y su tabla técnica de tokens. Integrar Usuario como modelo autenticable, resolver nombres de contraseña personalizados y mapear roles de la base. Estas tareas son de implementación posterior. Las rutas Blade existentes no se vuelven **API** por renombrarlas: deben devolver **JSON**, códigos y errores acordados.